

Rapport de projet Java

La Fabrique de Lunettes

Master 1 MIAGE - IDMC - Année universitaire 2025-2026
Programmation avancée



Réalisé par	Sidi SYLLA Salmane CHAHIDI Alex HUANG Abdullah ALI
Encadrant	Jérôme BERTRAND
Dépôt du projet	github.com/huang86u/Fabrique-Lunettes
Livrables	Application JavaFX, backend Java, README, JARs Maven, CI/CD et rapport

1. Résumé court

Ce rapport présente le projet Java intitulé La Fabrique de Lunettes. Le projet consiste à développer un système qui permet à l'utilisateur de commander des lunettes depuis une application JavaFX, puis de suivre leur commande jusqu'à obtenir un numéro de série. Le frontend n'appelle pas directement l'usine, mais garde une communication constante avec le backend Java à l'aide d'un broker MQTT. Ce choix technique nous permet de mettre en place une communication par événements, plus proche d'un système industriel où les acteurs ne travaillent pas toujours au même rythme.

Le frontend de l'application JavaFX c'est un écran d'accueil, un catalogue, un écran de commande, un suivi de statut, un affichage des numéros de série et une fonction de vérification et le backend reçoit les commandes, les valide, déclenche la fabrication, enregistre les numéros de série en mémoire et ensuite renvoie les informations au client. L'usine utilise un fabricant fourni sous une forme de dépendance Maven. Le projet est organisé en plusieurs modules Maven et possède un workflow GitHub Actions pour le packaging et la publication des livrables.

Avant de penser au résultat de ce projet, pour nous le projet nous a permis de travailler plusieurs compétences importantes en programmation avancée par exemple l'architecture orientée événements, une communication MQTT, un découpage en modules, une gestion de la concurrence, un packaging Maven, des tests et intégration continue. Le projet possède aussi des limites de la solution, par exemple l'absence d'une persistance, la sécurité MQTT minimale ou encore la couverture de tests encore peut être encore être améliorer. Les limites que nous avons vu sont présentées comme des pistes d'amélioration pour rendre le système plus complet dans un contexte de production réel.

Mots-clés : Java, JavaFX, MQTT, Maven, backend, frontend, usine, fabricant, CI/CD, GitHub Actions, architecture orientée événements.

2. Remerciements

Nous remercions notre encadrant pour le sujet proposé et pour les contraintes techniques imposées. Ces contraintes nous ont obligés à réfléchir profondément à une architecture plus sérieuse qu'une simple application locale, vu qu'on doit séparer le frontend et le backend et aussi l'usine.

Nous remercions également les différents membres de l'équipe pour la répartition du travail, les échanges réguliers et les tests effectués pendant le développement et aussi tout le travail fournis pour développer ce projet qui représente plus qu'un projet académique à nos yeux. Afin de bien mener le projet, fallait garder une communication et d'une bonne coordination entre plusieurs parties : interface JavaFX, serveur backend, usine de fabrication, communication MQTT et chaîne Maven. Chaque membre devait s'assurer qu'on était sur la même longueur d'onde et bien partager leur penser et l'effort pour le bon fonctionnement du système.

Contents

1. Résumé court	2
2. Remerciements	2
3. Introduction et contexte du projet.....	5
A. Objectif général du projet	6
B. Choix d'une architecture événementielle.....	6
4. Cahier des charges et besoins fonctionnels.....	7
A. Besoins fonctionnels côté utilisateur	7
B. Besoins techniques et contraintes.....	8
C. Traduction des besoins en exigences techniques	8
5. Organisation de l'équipe	9
6. Méthodologie de travail.....	10
A. Utilisation de Git et GitHub.....	10
B. Progression par validations successives	11
7. Architecture générale du système.....	12
A. Vue d'ensemble visuelle	13
B. Découplage des composants.....	13
8. Organisation Maven et structure du dépôt	14
A. Structure simplifiée	14
B. Lecture du dépôt par un nouvel arrivant.....	14
9. Module lunettes-shared	15
10. Frontend JavaFX	15
A. Illustration des écrans principaux.....	18
B. Gestion de l'état côté client.....	18
C. Lien entre interface et MQTT	19
11. Backend Java	19
A. Rôle fonctionnel du backend.....	19
B. Architecture générale.....	20
12. Usine et fabrication	20
A. Responsabilité de la classe Usine.....	20
B. Méthode de production attendue.....	21
C. Adaptation au fabricant fourni	21
D. Fabrication séquentielle par lots	21
E. Besoin de mutualisation.....	22
F. Mutualisation mise en œuvre dans Usine	22
G. Redistribution des produits.....	23

13.	Communication MQTT	24
A.	Justification de MQTT	24
B.	Démarrage et configuration du serveur	24
C.	Topics écoutés.....	24
D.	Topics publiés.....	25
E.	Déroulement d'une transaction (diagramme de séquence)	25
F.	Détection de la disponibilité	26
14.	Modélisation des échanges.....	26
A.	Module partagé.....	27
B.	Format de commande.....	27
C.	Format de livraison	28
D.	Validation d'une commande.....	28
E.	Cas d'erreur dans le scénario	28
15.	Gestion des statuts et numéros de série	29
A.	États de traitement.....	29
B.	Annulation et erreur de fabrication	30
C.	Numéros de série	30
D.	Cohérence entre interface et backend	32
16.	Tests et validation	33
A.	Commandes de test.....	33
B.	Différence entre tests unitaires et tests d'intégration	33
17.	Packaging Maven	34
A.	Intérêt du packaging pour le projet.....	34
B.	Reproductibilité du build.....	34
18.	CI/CD avec GitHub Actions.....	35
A.	Relation entre CI/CD et qualité	35
B.	Publication et secrets.....	35
19.	Difficultés rencontrées	36
A.	Apports de ces difficultés	36
B.	Gestion du temps et intégration finale.....	37
20.	Limites techniques et améliorations possibles.....	38
A.	Améliorations prioritaires	39
21.	Conclusion	40
A.	Bilan personnel et collectif	40
Annexes		41
Annexe		41

3. Introduction et contexte du projet

Dans le cadre du cours de programmation avancée Master 1 MIAGE à l'IDMC, il nous est demandé de développer un système de distribution pour une fabrique de lunettes connectées. La demande de départ est simple : juste un client qui doit pouvoir commander une ou plusieurs paires de lunettes, l'usine doit produire les modèles demandés, et ensuite le client doit recevoir les numéros de série des lunettes fabriquées. Derrière cette demande simple, le projet cache plusieurs points techniques importants et qu'on doit respecter à tout prix, comme la séparation des responsabilités, la communication entre applications différentes et la gestion d'un traitement qui ne se fait pas instantanément.

Le projet n'est donc pas juste de créer une interface graphique mais plutôt de construire un ensemble projet qui reste cohérent, avec un frontend JavaFX relié à un backend Java, et aussi d'une usine de fabrication et d'un broker MQTT. On peut voir qu'il y a plusieurs composants qu'on doit intégrer et chacun d'entre eux a un rôle précis. Le frontend pour l'utilisateur, le backend est responsable de la réception et du traitement des commandes, l'usine pour la partie métier de production et enfin le broker MQTT qui sert d'intermédiaire entre le client et le serveur.

Le contexte choisi est celui d'une entreprise qui veut digitaliser sa chaîne de commande et aussi celle de la fabrication. Dans une situation réelle, une commande passée par un client ne donne pas directement un résultat. La commande doit être reçue, validée, préparée, fabriquée, enfin livrée. Le projet reprend cette logique en la transposant dans une architecture logicielle. Le client envoie un événement de commande, le backend répond avec des événements de validation, de fabrication et de livraison, puis l'interface met à jour son affichage au fur et à mesure.

L'un des intérêts du projet est de forcer une réflexion sur l'asynchronisme. Dans une application classique, on pourrait imaginer un bouton qui appelle directement une méthode du backend et attendre une réponse. Mais là, ce modèle n'est pas adapté vu que la fabrication peut prendre du temps, plusieurs clients peuvent très bien commander en même temps et l'usine doit garder son propre rythme. Le choix d'une communication par événements leur permet de mieux représenter cette réalité. Le client n'est donc pas bloqué devant une réponse immédiate et il reçoit des informations dès qu'elles sont publiées par le backend.

Ce rapport présente donc à la fois le résultat obtenu et les choix de conception que nous avons faits pour le projet. Par exemple, il détaille le cahier des charges, l'organisation de l'équipe, la méthodologie de travail, l'architecture globale, la structure Maven, le fonctionnement du frontend, du backend et de l'usine, ainsi que la communication MQTT. Il y a aussi les tests, le packaging, la chaîne CI/CD, les difficultés rencontrées et les limites techniques du projet. L'objectif n'est pas seulement de montrer que l'application fonctionne, mais aussi d'expliquer comment elle fonctionne et pourquoi certains choix ont été faits.

A. Objectif général du projet

L'objectif principal était de livrer un système complet et exécutable, capable de couvrir le parcours d'une commande depuis l'interface utilisateur jusqu'à la fabrication. L'utilisateur doit être en mesure de choisir ses modèles de lunettes, choisir les quantités qu'il veut, envoyer la commande, suivre sa commande et consulter les numéros de série reçus. Le backend doit être capable de recevoir cette commande, il ne doit pas seulement la recevoir et de la traduire dans un format exploitable par l'usine, puis de renvoyer les informations nécessaires au frontend.

Le projet doit aussi respecter une logique de production ready. Ça veut dire que le code ne doit pas juste rester au stade d'un simple prototype lancé uniquement depuis un IDE, donc le projet doit être fait avec Maven, générer des JARs et proposer une chaîne de publication automatisée.

B. Choix d'une architecture événementielle

Nous avons choisi l'architecture événementielle parce qu'on pense que c'est celle ci qui correspond le plus à la nature du problème. Par exemple, une validation est un autre événement, la fin de fabrication et la livraison sont aussi des événements donc au lieu de faire dépendre directement le client du backend, chaque composant envoie ou reçoit des messages.

Dans notre solution, le broker MQTT est l'intermédiaire central, il gère la communication entre le backend et le frontend, il reçoit les messages du frontend et les transmet au backend, ensuite il transmet les réponses du backend au frontend. Le client n'a pas besoin de connaître la classe Java du serveur ni l'organisation interne de l'usine et tout ce qui est mis en place, il se contente d'envoyer et de recevoir des messages pour commander et suivre.

4. Cahier des charges et besoins fonctionnels

Le cahier des charges demande de développer un système qui permet de commander des lunettes, de gérer leur fabrication et d'assurer leur livraison. Afin de gérer cette demande, nous avons besoin de deux composants principaux : un frontend JavaFX pour le client et un backend Java pour gérer les commandes. La communication entre les deux ne doit pas se faire par appel direct, mais par événements à travers un broker MQTT. Cette contrainte donne au projet une dimension plus réaliste par rapport à tout ce qu'on avait fait en cours de java, vu qu'elle nous oblige à penser les échanges comme des messages indépendants.

La partie frontend doit être composée de plusieurs écrans. Le premier écran est un écran d'accueil qui présente l'application sans se compliquer la tâche, le client doit comprendre direct, le deuxième permet de commander les lunettes, le troisième affiche les numéros de série des lunettes fabriquées, le quatrième permet de vérifier un numéro de série, tous ces écrans sont le point les plus important pour l'utilisateur principal. Ils doivent être le plus simples possible pour que l'utilisateur comprenne directement ce qu'il peut faire et comment le faire.

La partie backend doit réceptionner les commandes et produire les lunettes demandées. Donc, le backend gère une usine qui utilise un fabricant fourni, le sujet nous impose une seule usine et un seul fabricant par usine, mais l'usine doit pouvoir gérer plusieurs commandes, ce qui veut dire qu'on doit aussi réfléchir à la concurrence et à la manière dont les commandes sont traitées lorsque plusieurs clients interagissent avec le système.

Un autre besoin important concerne les livrables. Le projet doit fournir des JARs exécutables, un README d'utilisation, un rapport détaillé et des ressources complémentaires comme les tests ou la CI/CD. Le rapport doit expliquer l'architecture, ce qui a été implémenté, ce qui manque ou marche pas, la répartition du travail entre les membres. Donc ce n'est pas juste du développement de code mais aussi d'avoir une vision sur l'ensemble du projet.

Dans le projet nous avons réalisé les fonctionnalités principales demandées. L'application JavaFX permet de naviguer entre les différents écrans, de commander des modèles de lunettes qu'on souhaite, bien sûr faut regarder le catalogue vu qu'on a un répertoire de modèle limité pour respecter le sujet, on peut suivre une commande et consulter les numéros de série. Le backend, il reçoit les commandes via MQTT, valide les quantités, déclenche la fabrication et publie les statuts. L'usine est intégrée dans le backend et s'appuie sur le fabricant fourni. L'ensemble est structuré avec Maven et dispose d'un workflow GitHub Actions pour la publication des livrables.

A. Besoins fonctionnels côté utilisateur

L'utilisateur doit être en mesure de commander sans connaître la complexité technique du système, donc l'interface doit masquer le broker MQTT, le backend, l'usine et le fabricant et tout le travail complexe que nous avons fourni. L'utilisateur doit tout simplement juste de voir un catalogue, choisir les lunettes qu'il veut et aussi la quantité qu'il désire et pour finir envoyer sa commande. Une fois la commande envoyée, il doit recevoir des notifications ou des indications pour suivre l'état de la commande. Ainsi on rend l'application aussi simple que même une personne qui n'a pas de connaissance technique et informatique peut l'utiliser.

L'utilisateur doit aussi pouvoir retrouver les numéros de série après la fabrication. Cette fonctionnalité est importante car elle matérialise la livraison. Dans le projet, une paire de lunettes fabriquée doivent être identifiées par un numéro de série. Ce numéro peut ensuite être vérifié dans l'écran qui correspond, ce qui renforce la cohérence du parcours.

B. Besoins techniques et contraintes

Les contraintes techniques sont toutes aussi importantes que les fonctionnalités. Le projet doit utiliser Java, JavaFX, Maven et MQTT, on doit séparer le frontend et le backend, les éléments partagés, on doit aussi intégrer une dépendance externe pour le fabricant. Ces contraintes obligent à gérer les dépendances de la bonne façon, les packages, la configuration et le lancement des applications.

La communication asynchrone est la contrainte la plus structurante. Elle nous oblige à concevoir un protocole de messages : quels topics utiliser, quels payloads envoyer, comment reconnaître une commande, comment publier un statut et comment répondre à une demande de vérification. Cette partie est très importante vu qu'une mauvaise définition des topics peut rendre le système difficile à maintenir.

C. Traduction des besoins en exigences techniques

Pour passer du cahier des charges au développement, nous avons traduit les besoins du sujet en une architecture claire qu'on va utiliser pour coder et développer l'application. Au début, le sujet parle de commander, fabriquer et livrer des lunettes enfin les grandes idées principales, mais le code c'est différent, il doit traduire ces idées en objets, messages, classes et workflows. Une commande devient un objet `Commande`, un modèle devient un `TypeLunette`, une livraison devient un message MQTT avec des numéros de série. Cette traduction permet de passer d'un besoin métier à une solution logicielle d'où l'importance de cette étape, une mauvaise traduction peut mener à un hors sujet.

Un autre point important, c'est l'attente de la livraison, l'application ne doit pas se bloquer après le clic sur le bouton de commande, le frontend doit aussi rester actif et continuer à fonctionner et recevoir les différentes mises à jour envoyées par le système. Cette contrainte a influencé notre conception de l'interface avec les écrans de suivi pour suivre l'état de la commande au fur et à mesure des événements reçus.

Enfin, le projet doit aussi prendre en compte la possibilité d'avoir plusieurs clients ou commandes en même temps donc pour éviter toute confusion et problème entre les échanges, chaque commande doit être liée à un identifiant unique, comme ça on peut différencier les réponses envoyées par le système, même si plusieurs commandes sont traitées en même temps, y'aura aucun problème. Même si l'application limite l'usage à une commande à la fois par client, cette organisation rend le système plus fiable et plus clair dans la gestion des communications.

5. Organisation de l'équipe

L'organisation de l'équipe, nous avons préféré de séparer par domaines au lieu de donner des petites tâches séparées où on risque de mélanger et provoquer de la confusion. Cette méthode nous a permis de chacun de travailler sur une partie du projet bien sûr tout en restant en lien avec les autres. L'idée était d'éviter que chacun développe une classe ou une fonctionnalité de son côté sans comprendre le fonctionnement global.

Salmane et Abdullah se sont principalement occupés du frontend JavaFX. Abdullah a fourni une version basique à Salmane pour qu'il l'améliore davantage, donc le responsable de cette partie est Salmane, son travail a consisté à créer l'interface utilisateur, organiser les différents écrans et gérer les actions de l'utilisateur. Donc il a mis en place les différents écrans, on a un écran d'accueil, le catalogue, le panier, l'écran de commande, le suivi du statut, l'affichage des numéros de série et l'écran de vérification. Le frontend devait aussi communiquer avec le backend grâce à MQTT pour envoyer les commandes et de recevoir les informations sur leur avancement.

Alex a pris en charge la partie backend et la logique générale du système mais c'est aussi le chef d'équipe qui a su guider le groupe de la bonne façon. Son rôle était de gérer le serveur Java, de recevoir les commandes envoyées par MQTT et ensuite d'organiser leur traitement avec l'usine. Le backend occupe une place centrale dans le projet vu qu'il fait le lien entre l'interface et la fabrication des lunettes, il ne faut pas oublier qu'il doit aussi suivre les différents états de la commande, comme la validation, la fabrication puis la livraison.

Sidi s'est concentré sur la partie usine et fabrication, cette partie est le cœur du projet, car elle est liée directement à la production des lunettes. Son travail c'est l'intégration du fabricant fourni, la génération des lunettes, la récupération des numéros de série et la simulation de la production. Il a aussi participé au packaging de l'usine dans le backend et aussi la publication Maven avec GitHub Actions.

Abdullah a aussi joué un rôle plus transversal sur la communication, le DevOps et la qualité du projet, cette partie y a la mise en place du broker MQTT, les échanges entre le frontend et le backend, les tests, l'organisation Maven globale et la CI/CD. Le but de son rôle était de relier toutes les parties du projet.

6. Méthodologie de travail

Pour ce projet, on a avancé de manière progressive petit à petit, on n'a pas essayé d'attaquer directement le développement de l'application d'un seul coup car cela aurait rendu le projet plus difficile à gérer et surtout on aurait été perdu très vite dans la confusion, on a donc préféré travailler par étapes en divisant le projet pour mieux suivre, d'abord on a cherché à bien comprendre le sujet et les demandes. Ensuite, on a traduit ces besoins en plan de développement de code, comme les classes, les objets, les messages MQTT ou encore les différents modules du projet.

Dès le départ, on a séparé le projet en plusieurs parties principales : un module partagé, un module frontend et un module backend. Ce découpage nous a permis d'avoir une organisation plus claire d'où son importance. Par exemple, le module partagé, c'est les éléments communs utilisés par plusieurs parties du projet, tandis que le frontend, c'est l'interface utilisateur et que le backend, son rôle est de gérer la logique de traitement des commandes, avec cette séparation, chaque partie avait un rôle précis, c'est ce qui a facilité le travail en groupe.

La deuxième étape a été de vérifier la communication entre les différentes parties, avant même de développer une interface complète du frontend, il fallait s'assurer que le frontend et le backend pouvaient bel et bien échanger des messages avec MQTT. C'était une étape importante, vu que la communication est au centre du projet donc si le broker, les topics ou le format des messages ne fonctionnent pas correctement, alors l'application ne marchera pas comme on veut même si l'interface est incroyable.

Ensuite, on est passé à l'intégration de la partie métier, le backend devait recevoir une commande et ensuite la lire puis la vérifier et enfin de transmettre les informations nécessaires à l'usine. De son côté, l'usine devait utiliser le fabricant fourni pour produire les lunettes et récupérer les numéros de série, il fallait aussi gérer les types de lunettes, les quantités demandées et les informations renvoyées après la fabrication.

La quatrième étape, c'est l'amélioration de l'expérience utilisateur, le frontend JavaFX ne doit pas seulement permettre d'envoyer une commande mais aussi d'afficher le suivi de la commande, l'historique, les numéros de série et l'écran de vérification. Et comme les réponses du backend arrivent de manière asynchrone avec MQTT, l'interface devait rester active et se mettre à jour quand un nouveau message était reçu donc fallait accorder beaucoup d'importance à cette partie pour bien gérer l'état de l'application côté client.

Enfin, on a préparé le projet pour la livraison. On a utilisé Maven pour compiler et packager les différents modules. GitHub Actions a aussi été configuré pour automatiser certaines étapes. De plus, ajouté des tests unitaires pour vérifier certaines classes partagées et des règles de validation côté backend. Cette étape permet de montrer que le projet ne se limite pas à une simple démonstration locale, mais qu'il est organisé de manière plus complète.

A. Utilisation de Git et GitHub

GitHub a été utilisé pour centraliser le code et faciliter le travail en groupe, tout le code était push dans GitHub pour que chaque puisse suivre ce qui était codé et ce qui ne l'était pas et le dépôt contient les différents modules Maven, les ressources, les workflows CI/CD et le README. Avec Git, on pouvait suivre l'évolution du projet, voir les modifications réalisées et revenir en arrière si besoin. Dans un projet de développement de code à plusieurs, c'est un

outil fantastique, il nous permet d'éviter de perdre du code et permet de mieux gérer les contributions de chacun.

B. Progression par validations successives

On a aussi choisi d'avancer par validations successives, c'est-à-dire qu'au lieu d'attendre la fin du projet pour vérifier si tout marchait, à chaque fois qu'on codait du code, on le testait directement et s'il marchait alors on le pushait sur GitHub. On a d'abord validé la structure Maven, puis la communication MQTT. Ensuite, on a relié progressivement cette communication à l'interface JavaFX et à l'usine. Cette méthode était plus sûre, car elle permettait de repérer les problèmes plus tôt.

Cette façon de travailler a aussi aidé à mieux répartir les tâches. Pendant qu'une partie de l'équipe avançait sur l'interface, une autre pouvait travailler sur le backend ou sur l'usine. Ensuite, on testait les points d'intégration ensemble pour vérifier que les différentes parties fonctionnaient bien entre elles. On peut dire que notre démarche était plutôt simple disons qu'on partait d'une version simple, on la testait, puis on ajoutait progressivement de nouvelles fonctionnalités.

Le choix de commencer par des échanges simples avec MQTT a été utile. Avant de gérer de vraies commandes, il fallait déjà être certain que le broker fonctionnait, que les clients pouvaient se connecter et que les messages arrivaient bien. Une fois cette base validée, on pouvait ajouter plus facilement les statuts, les numéros de série, l'historique et la vérification des commandes.

7. Architecture générale du système

L'architecture globale se fonde sur quatre composantes majeures : le frontend JavaFX, le broker MQTT, le backend Java et l'usine. Le module partagé est également important, vu qu'il contient les éléments communs employés par les deux applications. Cette structure favorise la réduction des dépendances superflues. L'interface utilisateur n'est pas au courant de la mise en œuvre de l'usine. Le backend n'est pas informé de la structure interne de l'interface. Ils ne s'accordent que sur des messages et des modèles communs.

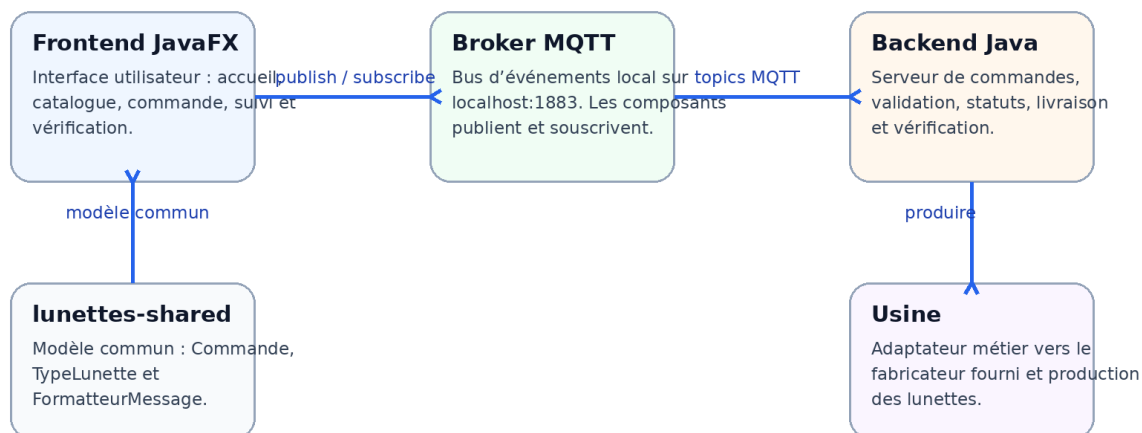
Le frontend est l'application que l'utilisateur peut voir. Il lui offre la possibilité de parcourir le catalogue, de sélectionner les quantités, de commander et de surveiller son statut. Quand une commande est effectuée, le frontend n'appelle pas directement une méthode du backend. Il envoie un message sur un topic MQTT. Le broker reçoit cette publication et se charge par la suite de diffuser le message aux composants qui se sont reliés.

Le broker MQTT joue le rôle d'intermédiaire. Il est mis en place localement dans notre projet à l'adresse localhost :1883. Il autorise l'implémentation du modèle publish/subscribe. Un composant diffuse un message sur un topic, et un autre composant qui est abonné à ce même topic le capture. Cette approche diminue le degré d'interdépendance entre les applications. Le client n'a pas besoin de savoir si le backend est une classe Java, un service distant ou une autre application. Il se contente de publier une commande.

Le backend est en charge de recevoir les commandes, de les valider et de gérer l'usine. Par la suite, il émet des notifications vers l'interface utilisateur : confirmation, voire une annulation, état de production, livraison et erreurs. Il se trouve au cœur du système donc son rôle est crucial pour le bien du projet. Il se doit de comprendre le protocole MQTT, mais aussi la logique liée à la commande et à la production. Donc, il joue le double rôle d'un serveur de communication et d'un orchestrateur métier.

L'usine est la partie qui transforme une commande en lunettes produites. Elle fonctionne comme un adaptateur vers le fabricant fourni. Cette séparation est utile car elle évite de mélanger directement la logique MQTT avec l'utilisation du fabricant. Le backend demande à l'usine de produire, et ensuite l'usine communique avec la dépendance externe. Après la fabrication des lunettes, le backend récupère les numéros de série et les envoie au client.

Architecture générale - Fabrique de Lunettes



Principe : le client ne contacte jamais directement l'usine. Les échanges passent par des événements MQTT.

Figure 1 - Architecture générale du système.

A. Vue d'ensemble visuelle

Le schéma ci-dessous présente un aperçu de la logique générale. Le client JavaFX envoie une commande au broker MQTT. L'événement est réceptionné par le backend, la commande est validée et l'usine est alors mise en marche. Une fois la fabrication terminée, le backend publie les informations de livraison. Le frontend reçoit alors les numéros de série et met à jour l'interface.

B. Découplage des composants

L'un des principes fondamentaux de l'architecture est le découplage. Le frontend n'a pas besoin d'importer les classes du backend, le backend n'a pas besoin de connaître les détails de l'interface même pas un peu. L'usine n'a pas besoin de savoir comment l'utilisateur a passé commande. Chaque partie se base uniquement sur ce qui lui est indispensable. Cela rend le système plus compréhensible et plus facile à faire évoluer.

Ce découplage rend aussi les tests plus simples. Il n'est pas obligatoire de réécrire l'usine si on veut modifier l'interface JavaFX. Si on veut modifier la manière de fabriquer les lunettes, le frontend peut rester toujours la même tant que les messages de retour gardent le même sens et forme dans le cas contraire, il va falloir s'adapter.

Dans une version future, ce même principe pourrait être utilisé pour ajouter d'autres clients. Par exemple, on pourrait imaginer une interface web ou une application mobile qui envoie les mêmes instructions MQTT. Le backend n'aurait pas besoin de savoir si la commande vient de JavaFX ou d'un autre client. Cette possibilité montre l'importance d'un protocole de communication clairement établi.

8. Organisation Maven et structure du dépôt

Le projet est organisé comme un projet Maven multi-modules. On pense que ce choix est la bonne puisqu'elle facilite la division des composants du système tout en maintenant une administration centralisée des dépendances. Le fichier pom.xml établit la structure générale, tandis que chaque module joue le rôle qui lui est accordé. Cette organisation facilite la compilation, le test et le packaging.

Le module lunettes-shared contient les classes communes ou si vous préférez les classes partagées. Il est utilisé à la fois par le frontend et par le backend. Ce module nous permet d'éviter d'avoir des doublons des modèles de commande ou les types de lunettes.

L'application graphique se trouve dans le module lunettes-javafx. Il regroupe toutes les classes de démarrage, la gestion de l'interface utilisateur, la configuration du frontend et aussi le service MQTT côté client. C'est dans ce module qu'on trouve la logique d'affichage et d'interaction utilisateur. Il se base sur le module partagé pour construire les commandes et utilise le client MQTT pour communiquer avec le broker.

Le module lunettes-backend contient le serveur Java, la logique de validation, la gestion des messages MQTT et aussi l'intégration avec l'usine. Il se base aussi sur le module partagé dans le but de décoder les commandes et du fabricant fourni pour fabriquer les lunettes. Ce module est packagé sous forme de JAR exécutable avec ses dépendances qu'il faut prendre en compte, et c'est ce qui permet de le lancer indépendamment du frontend.

A. Structure simplifiée

On peut résumer la structure utile du dépôt de la manière suivante, le fichier pom.xml se trouve à la racine du projet. Les modules lunettes-shared et lunettes-javafx et enfin lunettes-backend contiennent respectivement le modèle commun, le frontend et le backend. Le dossier assets contient les éléments visuels. Le dossier github/workflows contient le flux de travail pour la publication. Cette structure est lisible et facile à comprendre et c'est ce qui rend le projet par une personne qui n'a pas contribué ou impliqué dans le projet.

B. Lecture du dépôt par un nouvel arrivant

La structure du dépôt doit faciliter la compréhension rapide du projet pour une personne qui ne le connaît pas, une personne extérieure. L'utilisation de modules clairs est très utile : shared pour les éléments communs, javafx pour l'interface utilisateur, backend pour le serveur. Pour un projet d'équipe, il faut avoir une logique claire et simple puisque le code doit pouvoir être repris, corrigé et évalué sans se baser uniquement sur des explications orales des développeurs.

Le README joue aussi un rôle dans cette compréhension. Il doit expliquer comment installer les prérequis, démarrer le broker, compiler le projet et lancer les JARs. Les explications sont nécessaires même si le code marche à 100%, et un projet qui a du mal à démarrer perd en qualité. La documentation d'exécution fait donc partie du livrable technique.

L'utilisation de Maven permet d'éviter les configurations manuelles différentes selon les postes. Les dépendances sont déclarées dans les fichiers pom.xml et tous les membres ont les mêmes commandes de compilation. Tout ça permet de minimiser les erreurs du type « cela fonctionne sur mon ordinateur mais pas chez les autres ».

9. Module lunettes-shared

Le module lunettes-shared joue un rôle assez discret mais il reste tout de même important. Dans une structure répartie, les éléments ont besoin d'un langage partagé, le frontend doit être en pouvoir de formuler une commande d'une manière que le backend peut comprendre et le backend doit pouvoir décoder cette commande sans dépendre de classes graphiques. Le module partagé doit satisfaire les besoins en regroupant les composants communs aux deux applications, et d'avoir un échange entre le frontend et le backend sans poser de problème.

La classe Commande représente une commande métier, elle contient les lignes de commande et offre aussi la possibilité de calcul total. Ce type de classe est utile puisqu'elle évite de manipuler uniquement les chaînes de caractères tout au long du projet. Même si les messages MQTT sont envoyés sous forme de payload textuel, le code java les manipule par la suite sous forme d'objets plus organisés. Cela améliore la clarté et diminue les fautes.

L'énumération TypeLunette définit les modèles disponibles. L'analyse du code a permis d'identifier les modèles suivants : BANANA, CHATGPT, LE_CHAT et CLAUDE. On a choisi d'utiliser une énumération puisqu'elle permet de définir clairement les valeurs autorisées. C'est plus sûr que de simples chaînes de caractères, où une faute de frappe pourrait créer une erreur compliquée à trouver ou détecter. En utilisant une énumération, le code devient plus propre, mieux structuré et plus simple à modifier si un nouveau modèle doit être ajouté.

La classe FormateurMessage est responsable de l'encodage et du décodage des messages MQTT. Elle convertit une commande en chaîne de caractères, puis facilite la reconstruction d'une commande à partir du payload qu'elle a reçu. Dans le projet, nous avons choisi un format simple, par exemple BANANA :2, CLAUDE :1. Qui reste un format assez simple à lire et à tester, mais il faut dire qu'il est moins solide qu'un format structuré comme celui de JSON. Cette limite sera abordée dans la partie qui sera consacrée aux améliorations possibles.

Le module partagé représente un bon équilibre entre simplicité et cohérence. Il n'implique pas de logique graphique et n'assume pas la direction de l'usine. Son rôle est d'identifier les objets et les formats communs. Cette séparation est importante car elle permet d'éviter de créer une dépendance directe entre le frontend et le backend. Les deux modules peuvent évoluer séparément tant qu'ils respectent les mêmes modèles.

10. Frontend JavaFX

Le frontend JavaFX est la partie visible du projet. Il doit permettre à l'utilisateur de réaliser tout le parcours de commande sans avoir besoin de comprendre le backend ou MQTT. L'interface

est organisée avec un menu et plusieurs autres pages. Cette organisation donne une application claire. Il y a accueil, catalogue, commande, statut, numéros de série et vérification.

La page d'accueil présente l'application et indique l'état de connexion au broker MQTT. Si le broker n'est pas disponible, l'utilisateur peut comprendre que l'application ne pourra pas échanger correctement avec le backend. L'accueil propose aussi des accès rapides vers le catalogue et vers les numéros de série. Cela donne une entrée simple dans l'application.

Le catalogue présente les modèles disponibles. Chaque modèle possède une image, un nom, une description, un prix et un sélecteur de quantité. L'utilisateur peut donc composer sa commande de manière visuelle. Les quantités sont limitées par l'interface, ce qui évite une partie des erreurs avant même l'envoi au backend. Cette validation côté interface ne remplace pas la validation backend, mais elle améliore l'expérience utilisateur.

Le panier récapitule les produits sélectionnés avant l'envoi. C'est une étape importante car elle permet à l'utilisateur de vérifier sa commande. Le bouton de validation envoie ensuite la commande via le MQTT du frontend. À partir de ce moment, l'interface doit gérer une logique asynchrone : la commande est envoyée, mais la fabrication ne se termine pas immédiatement. L'écran de commande sert donc à suivre le statut de la commande.

L'écran de suivi montre les différentes étapes de la commande. On retrouve l'envoi, la validation, la fabrication et l'expédition. Cette présentation rend visible un processus qui est réparti entre plusieurs composants. L'utilisateur voit seulement l'état métier, alors que l'application reçoit en réalité plusieurs messages MQTT séparés. Cela montre une bonne traduction entre un flux technique et une information compréhensible.

L'écran des numéros de série affiche les références reçues après la livraison. Cette partie confirme que la fabrication a bien produit des objets identifiables. Chaque numéro peut ensuite être utilisé dans l'écran de vérification. La vérification permet de saisir un numéro de série et de demander au backend s'il correspond à une paire fabriquée. Cette fonctionnalité complète le cycle de vie de la commande.

Dans le code, la classe ClientShell concentre une grande partie de l'interface et des interactions. Cette solution a l'avantage de rendre l'application rapide à développer et de centraliser la navigation. Cependant, elle peut devenir plus difficile à maintenir si l'application grossit. Il serait possible de découper l'interface en contrôleurs ou composants plus spécialisés.

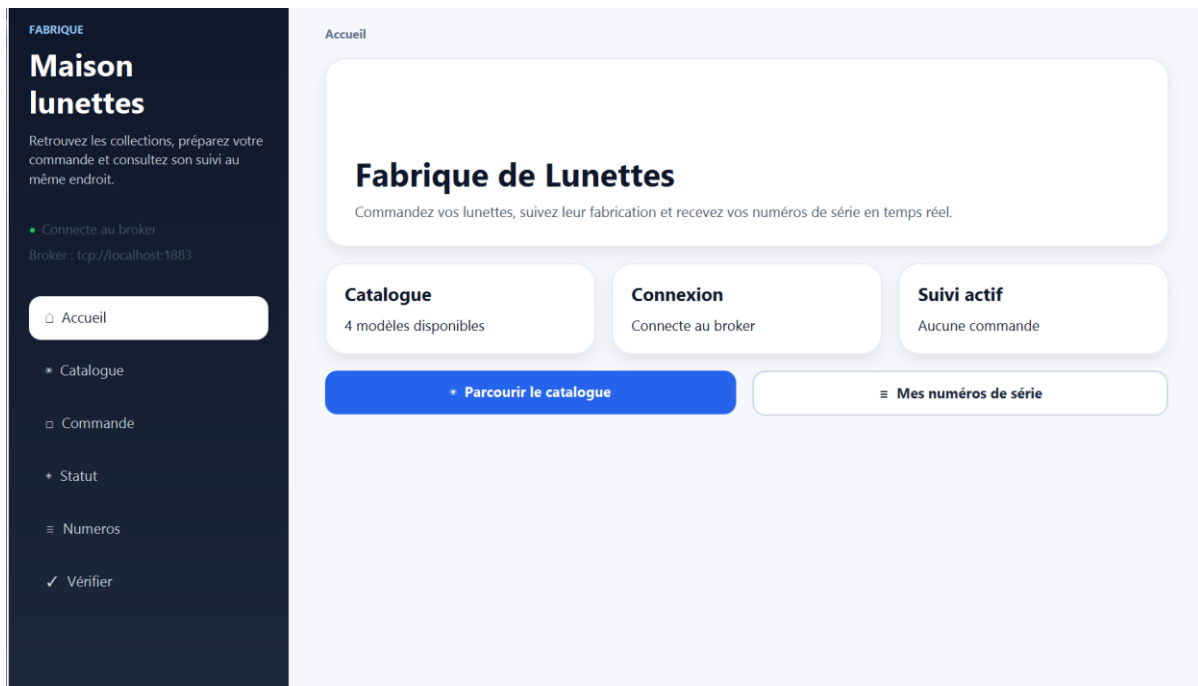


Figure 2 - Écran d'accueil avec état de connexion au broker MQTT.

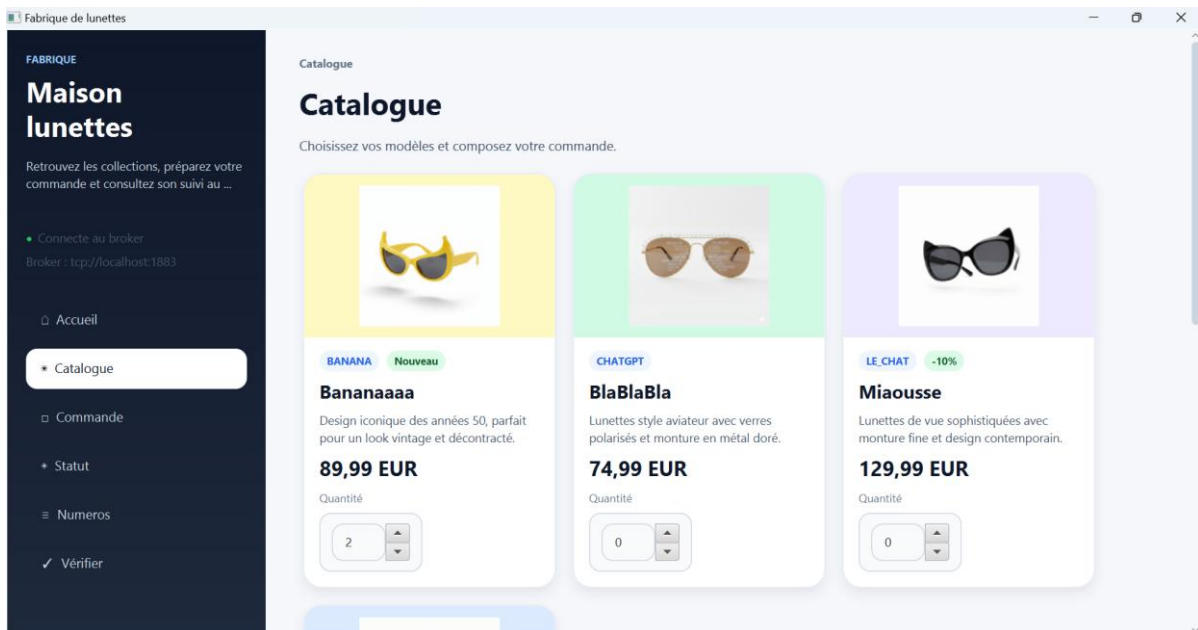


Figure 3 - Catalogue des modèles disponibles et sélection des quantités.

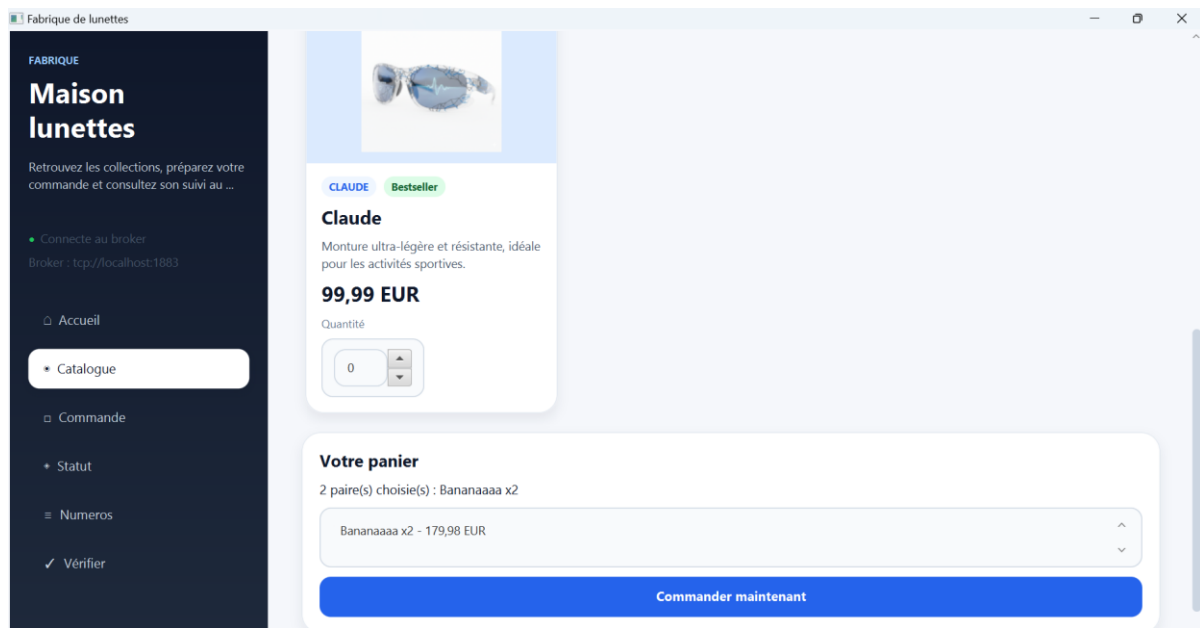


Figure 4 - Panier utilisateur avant validation de la commande.

A. Illustration des écrans principaux

Les captures d'écran présentées ci-dessus illustrent les principales fonctionnalités de l'interface et permettent de visualiser le parcours complet de l'utilisateur. Elles montrent notamment la connexion au broker MQTT, la consultation du catalogue, la sélection des produits, la validation d'une commande, le suivi de fabrication ainsi que la consultation et la vérification des numéros de série. Ces écrans servent de support concret pour comprendre le fonctionnement global de l'application.

B. Gestion de l'état côté client

Le frontend ne se limite pas à afficher des boutons ou des informations statiques. Il doit gérer un état qui évolue continuellement en fonction des messages reçus depuis le backend. Une commande peut passer par plusieurs étapes : inexistante, envoyée, validée, en fabrication, fabriquée puis expédiée. De la même manière, les numéros de série ne sont pas disponibles immédiatement ; ils apparaissent uniquement après la fin de la production.

L'interface doit donc être capable de réagir dynamiquement aux événements MQTT tout en conservant les données déjà reçues. Cette logique explique la présence d'un historique des commandes et d'un écran de suivi détaillé. Même après la livraison, l'utilisateur peut revenir consulter sa commande et retrouver les numéros de série associés. Cela renforce la continuité de l'expérience utilisateur et évite que les informations disparaissent dès la réception d'un message.

L'ergonomie générale a également été pensée pour simplifier l'utilisation de l'application. Le menu latéral permet d'accéder rapidement aux différentes fonctionnalités, tandis que les cartes produits rendent le catalogue plus clair et plus agréable à parcourir. Les boutons d'action sont visibles et les indicateurs de connexion permettent de savoir immédiatement si le système est disponible. Ces choix ne sont pas uniquement esthétiques : ils réduisent les risques d'erreur et améliorent la compréhension du fonctionnement de l'application.

C. Lien entre interface et MQTT

La communication MQTT est centralisée dans un service dédié côté frontend. Cette approche permet de séparer clairement la logique de communication réseau de l’affichage graphique. L’interface utilise ce service pour envoyer des commandes ou effectuer des vérifications de numéros de série, tandis que le service MQTT écoute les réponses du backend et déclenche les mises à jour nécessaires.

Cette séparation rend le code plus propre et plus maintenable qu’une solution où chaque bouton interagirait directement avec les objets MQTT. Le frontend devient ainsi responsable de l’affichage, tandis que le service MQTT gère uniquement les échanges réseau.

Le frontend doit également prendre en compte la gestion des threads. En JavaFX, toutes les modifications visuelles doivent être effectuées sur le thread graphique principal. Or, les messages MQTT arrivent depuis des threads différents. Les mises à jour de l’interface doivent donc être réinjectées correctement dans le thread JavaFX afin d’éviter des erreurs d’affichage ou des problèmes de concurrence. Cette contrainte est classique dans les applications graphiques connectées à des services réseau asynchrones.

11. Backend Java

Le backend de la Fabrique de lunettes représente l’unité de production du système distribué. Il reçoit les demandes émises par l’application JavaFX, applique les règles fonctionnelles de validation, organise la production avec la bibliothèque fabricant, puis transmet les événements nécessaires au client. Sa conception est directement guidée par l’énoncé, qui impose une communication asynchrone et une séparation nette entre le serveur MQTT et l’usine.

Cette partie décrit le fonctionnement réellement mis en œuvre dans le projet. Elle présente l’architecture du backend, le pilotage de la fabrication, la mutualisation des appels concurrents, la communication MQTT, le format des messages, la gestion des résultats et les mécanismes de livraison. Elle expose également les tests, le packaging Maven, le workflow de publication et les limites qui doivent encore être améliorées.

A. Rôle fonctionnel du backend

L’application JavaFX permet à l’utilisateur de choisir les modèles de lunettes et d’envoyer une commande. Le backend doit recevoir cette commande, vérifier sa cohérence et lancer une fabrication. Une fois les lunettes produites, il renvoie les numéros de série associés afin que le client puisse afficher la livraison et contrôler ultérieurement la validité d’un produit.

Le backend n’est pas conçu comme un service HTTP. Le traitement d’une commande peut prendre plusieurs secondes, car chaque lunette est réellement produite par la bibliothèque fournie. Une réponse synchrone aurait obligé l’interface cliente à attendre la fin complète de la fabrication. Le modèle événementiel permet au client de recevoir successivement une validation, un statut de traitement et une livraison.

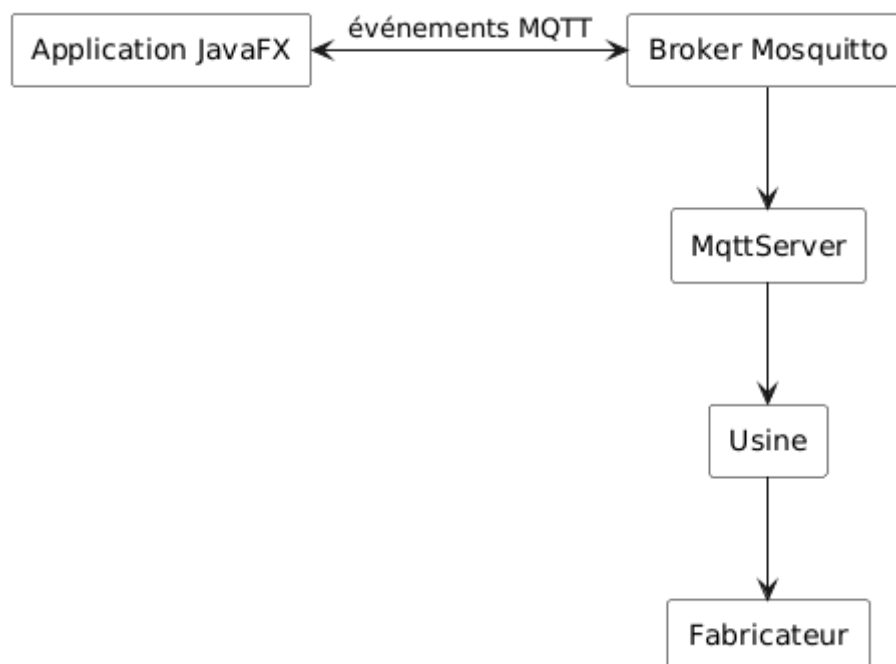
- Recevoir les commandes postées sur le bus MQTT
- Valider les quantités et les modèles demandés avant fabrication
- Déclencher la production en respectant la capacité disponible

- Mutualiser les commandes lorsque plusieurs threads appellent l'usine en parallèle
- Publier les événements de validation, de progression, de livraison ou d'erreur
- Enregistrer et vérifier les numéros de série produits
- Fournir un JAR prêt à être exécuté et publié automatiquement

B. Architecture générale

Le dépôt est organisé en trois modules Maven. Le module partagé contient les éléments nécessaires au dialogue entre le client et l'usine. Le module backend contient la logique de communication et de production. Le module JavaFX contient l'interface utilisateur. Cette organisation limite les dépendances et évite de dupliquer le protocole de message dans deux applications distinctes.

Module	Contenu principal	Responsabilité
lunettes-shared	Commande, TypeLunette, FormatteurMessage	Définir le modèle et le protocole communs
lunettes-backend	App, MqttServer, Usine	Recevoir, produire et livrer
lunettes-javafx	Interface et service MQTT client	Commander, suivre et vérifier



Dans cette architecture, MqttServer gère la communication extérieure tandis que Usine gère les contraintes de fabrication. Cette répartition respecte exactement la dépendance demandée dans le sujet, à savoir un serveur qui dépend de l'usine, elle-même dépendante de fabricant.

12. Usine et fabrication

A. Responsabilité de la classe Usine

La classe Usine est l'adaptateur entre le modèle du projet et la bibliothèque fabricant. Cette bibliothèque définit ses propres types de lunettes, possède une capacité de production et impose une phase de configuration avant la fabrication. Il est préférable de centraliser ces contraintes dans une seule classe plutôt que de les laisser apparaître dans le serveur MQTT.

Le serveur transmet une commande validée à l'usine. L'usine convertit alors les types partagés vers les types attendus par fabricant, organise les lots et retourne une liste de lunettes contenant leur type et leur numéro de série. Le serveur peut ensuite se concentrer sur la publication des messages MQTT.

B. Méthode de production attendue

L'énoncé impose un point d'entrée capable de recevoir une association entre les types et les quantités à produire. Le code propose la méthode utilisant les types de la bibliothèque fabricant ainsi qu'une surcharge utilisant l'objet Commande du module partagé. La surcharge évite que MqttServer réalise lui-même la conversion.

```
public List<Fabricateur.Lunette> produire(  
    final Map<Fabricateur.TypeLunette, Integer> typesLunettes)  
  
public List<Fabricateur.Lunette> produire(Commande commande)
```

La conversion repose sur le nom de chaque valeur de l'énumération. Par exemple, le type BANANA défini dans lunettes-shared est converti vers le type BANANA défini dans fabricant. Cette approche reste simple et sûre tant que les noms des modèles sont identiques dans les deux modules.

C. Adaptation au fabricant fourni

L'intégration du fabricant fourni oblige à respecter une interface existante. Le projet ne contrôle pas entièrement le comportement interne de cette dépendance. L'usine sert donc à adapter notre logique à cette contrainte. Elle reçoit une commande exprimée dans notre modèle, puis la convertit pour le fabricant. Cette couche d'adaptation est une bonne pratique lorsque l'on travaille avec une bibliothèque externe.

Cette approche protège aussi le reste du code. Si le fabricant change dans une version future, il sera plus simple de modifier la classe Usine que de corriger toutes les classes du backend. Le serveur MQTT n'a pas besoin de connaître les détails de configuration de la machine. Il demande seulement une production et attend un résultat exploitable.

D. Fabrication séquentielle par lots

La première étape demandée par le sujet consiste à respecter la capacité maximale de la machine. Une commande ne peut pas être fabriquée en un seul cycle si elle contient plus de lunettes que le nombre d'emplacements disponibles. L'usine calcule donc un lot, configure le

fabricateur, produit les lunettes de ce lot, puis répète l'opération tant que la commande n'est pas entièrement traitée.

Commande	Capacité de la machine	Cycles nécessaires
BANANA x 5	4	Un cycle de 4 puis un cycle de 1
CLAUDE x 2	4	Un cycle de 2
BANANA x 3 et CHATGPT x 3	4	Un premier cycle de 4 puis un second cycle de 2

Cette organisation garantit qu'aucune configuration transmise au fabricant ne dépasse sa capacité. Elle couvre le scénario initial du sujet et constitue la base nécessaire avant toute optimisation de productivité.

E. Besoin de mutualisation

Le sujet demande ensuite une optimisation. Lorsque plusieurs commandes arrivent en parallèle et que la capacité de la machine est suffisante, il est inutile de déclencher un cycle distinct pour chaque commande. La mutualisation consiste à regrouper ces demandes dans un même cycle tout en préservant les résultats individuels attendus par chaque client.

Cas	Sans mutualisation	Avec mutualisation
Trois commandes de deux lunettes, capacité de huit	Trois cycles de deux lunettes	Un cycle commun de six lunettes
Deux commandes de trois lunettes, capacité de quatre	Deux traitements indépendants	Un premier lot plein puis le reliquat dans un autre lot

F. Mutualisation mise en œuvre dans Usine

La formulation de l'énoncé précise que la méthode produire doit pouvoir être appelée par plusieurs threads simultanément. La mutualisation a donc été placée dans la classe Usine, et non dans le serveur MQTT. Chaque thread de commande appelle directement produire. L'usine reçoit ces appels dans une file interne de demandes.

Chaque demande contient les quantités à produire et une réponse asynchrone de type `CompletableFuture`. Le thread appelant reste en attente de cette réponse. En parallèle, un thread interne à l'usine attend la première demande, puis ouvre une fenêtre de regroupement de 200 millisecondes. Toutes les demandes déjà arrivées pendant cette fenêtre sont intégrées au même traitement.

- 1 Un thread associé à une commande appelle `Usine.produire`
- 2 La demande est ajoutée à la file interne de l'usine
- 3 Le thread de traitement récupère une première demande puis attend de millisecondes
- 4 Les demandes disponibles sont fusionnées par type de lunette
- 5 L'usine forme un lot conforme à la capacité du fabricant
- 6 Les lunettes du lot sont fabriquées en parallèle
- 7 Les résultats sont répartis selon les demandes initiales
- 8 Chaque appel produit reçoit sa liste de lunettes et le serveur publie sa livraison

Appel concurrent	Contenu demandé	Résultat retourné après mutualisation
Thread A	BANANA x 2	Deux lunettes BANANA
Thread B	CLAUDE x 2	Deux lunettes CLAUDE
Thread C	CHATGPT x 2	Deux lunettes CHATGPT
Cycle global	Six lunettes avec capacité de huit	Production commune dans le même lot

G. Redistribution des produits

Une fabrication mutualisée ne doit pas mélanger les livraisons visibles par les clients. Après la fabrication du lot, l'usine classe les lunettes obtenues dans des files par type. Pour chaque demande initiale, elle extrait le nombre de lunettes correspondant à chaque type commandé. Chaque Completable est ensuite complété avec la liste qui appartient uniquement à son appel.

Cette redistribution est indispensable. Sans elle, le gain de productivité obtenu par la fabrication commune rendrait impossible l'envoi d'une livraison correcte pour chaque identifiant de commande. Le serveur ne reçoit donc jamais un résultat global à démêler. Il récupère déjà le résultat individualisé fourni par l'usine.

La classe Usine prend en charge le regroupement et la redistribution car elle est la seule classe qui doit connaître la capacité et la logique de fabrication. MqttServer reste limité aux événements MQTT et au suivi des transactions.

13. Communication MQTT

A. Justification de MQTT

MQTT est un protocole léger construit autour du modèle publication et abonnement. Dans ce projet, il permet au client d'envoyer une commande sans établir de dépendance directe avec le backend. Le broker reçoit les messages puis les distribue aux composants abonnés. Cette organisation permet également à plusieurs clients d'utiliser le système au même moment.

La communication asynchrone correspond particulièrement bien au processus de fabrication. Une commande n'a pas une réponse unique immédiate. Elle évolue au fil de plusieurs événements, depuis sa validation jusqu'à la livraison ou jusqu'à une erreur terminale.

B. Démarrage et configuration du serveur

Le point d'entrée du backend est la classe App. Elle lit les propriétés embarquées dans les ressources, récupère l'adresse du broker et l'identifiant du client MQTT, puis démarre MqttServer. Ce fonctionnement permet de modifier la destination du broker sans intervenir sur le code de validation ou de fabrication.

Propriété	Valeur présente dans le projet	Utilité
mqtt.broker.url	tcp://localhost:1883	Adresse TCP du broker Mosquitto
mqtt.client.id	UsineBackend	Nom du client MQTT représentant le backend

Le broker utilise le port 1883 sans authentification et sans TLS, conformément au périmètre défini par l'énoncé. Après la connexion, MqttServer installe son callback de réception et s'abonne aux topics nécessaires.

C. Topics écoutés

Topic reçu	Charge utile	Action réalisée
orders/{identifiant}	Liste des types et quantités	Décoder, valider et traiter une commande
serials/{numéro}/check	Aucune donnée nécessaire	Vérifier l'existence d'un numéro de série
backend/ping	Identifiant du client	Répondre pour confirmer la présence du backend

Le serveur s'abonne à orders/+ avec un joker MQTT. Ce joker intercepte les nouvelles commandes, car leur identifiant est variable. Il ne capture pas les événements de livraison ou de statut qui possèdent un niveau supplémentaire dans leur topic.

D. Topics publiés

Topic publié	Donnée	Signification
orders/{id}/validated	Message vide	La commande respecte les règles et sera produite
orders/{id}/cancelled	Description de l'annulation	La commande est refusée avant fabrication
orders/{id}/status	processing ou processed	La production commence ou se termine
orders/{id}/delivery	Type et numéro de série	La commande est livrée avec succès
orders/{id}/error	Description de l'erreur	La fabrication a échoué
serials/{numéro}	Type ou invalid	Résultat d'une vérification de série
backend/pong	Identifiant reçu	Le backend répond au contrôle de présence

Les publications sont réalisées avec le niveau QoS 1. Le broker garantit alors une livraison au moins une fois. Ce niveau est préférable à QoS 0 pour des messages importants, même si une application plus complète devrait aussi prévoir le cas d'un message reçu en double.

E. Déroulement d'une transaction (diagramme de séquence)

Lorsqu'une commande arrive, MqttServer extrait son identifiant depuis le topic et démarre un thread dédié. Ce thread décode la commande puis applique les règles métier. Si la commande est acceptée, le serveur informe le client et appelle l'usine. Pendant cet appel, l'usine peut regrouper cette demande avec celles d'autres threads.

Le diagramme suivant présente la séquence principale sous forme simplifiée. Il montre le rôle du client, du broker, du backend et de l'usine. Chaque flèche correspond à un message ou à un appel métier.

Séquence simplifiée d'une commande

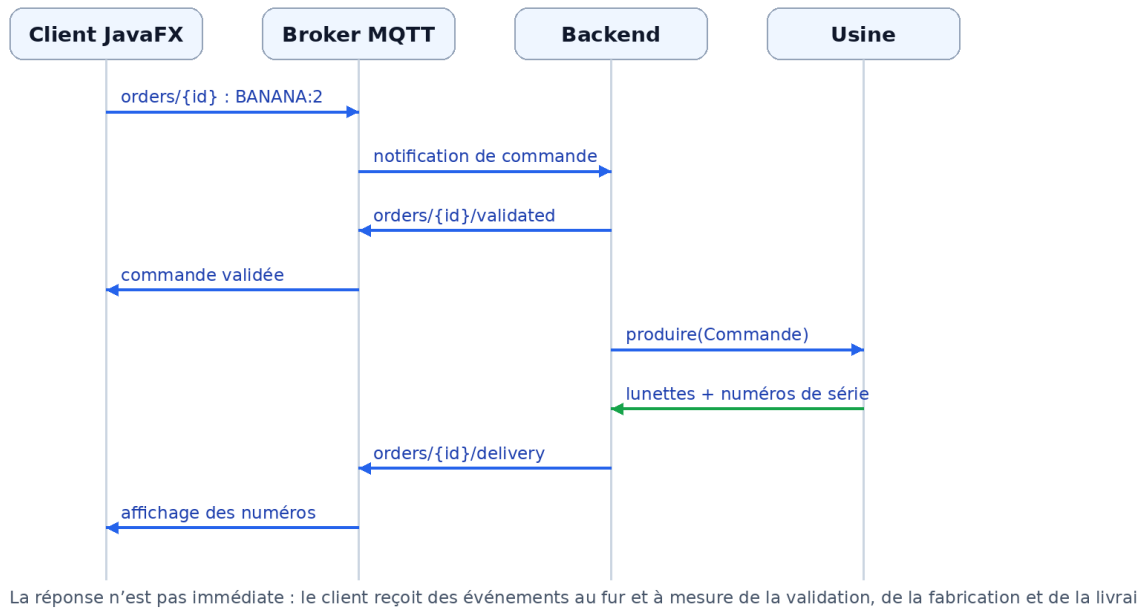


Figure 5 - Diagramme de séquence simplifié d'une commande.

F. Détection de la disponibilité

Une connexion au broker ne prouve pas qu'une usine est active. Un client pourrait réussir à publier sa commande sans qu'aucun backend ne l'écoute. Le projet ajoute un mécanisme de présence simple. Le frontend envoie un message sur backend/ping et attend un message équivalent sur backend/ping. En l'absence de réponse dans le délai configuré par le frontend, la commande n'est pas envoyée.

Ce choix apporte une première réponse au besoin de gestion d'une usine indisponible. Une solution plus complète pourrait publier régulièrement un état de présence ou exploiter les messages retained de MQTT.

14. Modélisation des échanges

La modélisation des échanges permet de comprendre le scénario complet d'une commande. Tout commence lorsque l'utilisateur sélectionne un ou plusieurs modèles dans le catalogue. Le frontend construit alors une commande avec un identifiant unique. Cette commande est encodée sous forme de message textuel, puis publiée sur le topic correspondant. À ce stade, le frontend ne sait pas encore si la fabrication va réussir. Il attend des événements de retour.

Le broker MQTT reçoit le message et le transmet au backend, car celui-ci est abonné aux commandes. Le backend décode le payload et vérifie la validité de la commande. Si la commande n'est pas valide, il publie un message d'annulation ou d'erreur. Si elle est valide, il

publie un message de validation et un statut indiquant que la commande est en cours de traitement. Cette première réponse rassure l'utilisateur : sa commande a bien été prise en compte.

Le backend appelle ensuite l'usine. L'usine convertit les types de lunettes et configure le fabricant. La fabrication produit des objets lunettes contenant des numéros de série. Une fois la fabrication terminée, le backend enregistre les numéros de série dans son registre mémoire. Cette étape est nécessaire pour permettre une vérification ultérieure. Sans enregistrement, le backend ne pourrait pas répondre à une demande de vérification.

Après la production, le backend publie un statut indiquant que la fabrication est terminée, puis un message de livraison contenant les numéros de série. Le frontend reçoit ces messages et met à jour l'écran de commande, l'historique et l'écran des numéros. L'utilisateur peut alors consulter les numéros et vérifier qu'un numéro précis est valide.

Ce scénario montre que le système fonctionne par étapes. Il ne s'agit pas d'une seule réponse globale, mais d'une succession de messages. Cette logique est plus complexe à programmer qu'un appel synchrone, car chaque événement peut arriver à un moment différent. En revanche, elle rend le système plus souple et plus réaliste pour une chaîne de fabrication.

A. Module partagé

Le module lunettes-shared formalise le contrat entre le frontend et le backend. Il contient le type `Commande`, l'énumération `TypeLunette` et la classe `FormateurMessage`. Les deux applications utilisent ainsi exactement les mêmes modèles et les mêmes règles d'encodage.

Classe	Rôle	Intérêt
Commande	Associer un type de lunette à une quantité	Transporter le contenu fonctionnel d'une commande
TypeLunette	Définir les quatre modèles disponibles	Éviter les valeurs arbitraires dans le code
FormateurMessage	Encoder et décoder les payloads	Assurer un protocole identique des deux côtés

B. Format de commande

L'énoncé impose de concevoir un format spécifique au lieu d'utiliser un format standard tel que JSON. Le projet utilise un format texte composé de couples `TYPE:QUANTITE` séparés par une virgule. Cette syntaxe est suffisante pour exprimer une commande et reste lisible durant les essais manuels.

Format	général
TYPE:QUANTITE,TYPE:QUANTITE	
Exemple	
BANANA:2,CLAUDE:1	

Le message d'exemple signifie que le client demande deux lunettes du type BANANA et une lunette du type CLAUDE. Lors du décodage, la chaîne devient une instance de `Commande` contenant une association entre chaque type et sa quantité.

C. Format de livraison

Le message de livraison conserve la même logique textuelle, mais la quantité est remplacée par le numéro de série retourné par fabricant. Le frontend peut afficher les séries produites et les réutiliser pour la fonction de contrôle.

Format	général
TYPE:NUMERO_SERIE,TYPE:NUMERO_SERIE	
Exemple	
BANANA:SN-001,CLAUDE:SN-002	

D. Validation d'une commande

La validation est réalisée par MqttServer avant tout appel à la production. Ce choix empêche l'usine d'utiliser des ressources pour une demande manifestement incohérente. Une commande annulée produit un événement cancelled et ne doit plus déclencher d'autre événement de production.

Règle prévue par le sujet	Réalisation observée	État
Le type doit être connu	Le décodeur reconnaît les valeurs de TypeLunette	Respecté
La quantité totale doit être strictement positive	Le total inférieur ou égal à zéro est refusé	Respecté
Chaque quantité doit être supérieure ou égale à zéro	Les quantités négatives sont refusées	Respecté
Chaque quantité doit être strictement inférieure à dix	Une quantité égale ou supérieure à dix est refusée	Respecté
Une commande refusée doit s'arrêter	Le serveur publie cancelled puis retourne immédiatement	Respecté

E. Cas d'erreur dans le scénario

Le scénario nominal montre le fonctionnement attendu, mais il faut aussi penser aux erreurs. Une commande peut être invalide, le backend peut rencontrer une exception pendant la fabrication ou un numéro de série peut être inconnu. Dans ces cas, le backend doit publier un message clair afin que le frontend n'attende pas indéfiniment. La présence de topics d'annulation et d'erreur répond à cette problématique.

Un autre cas concerne l'indisponibilité du backend. Le frontend peut être connecté au broker, mais le serveur métier peut être arrêté. Le mécanisme de ping permet alors de distinguer la connexion au broker de la disponibilité du backend. Cette différence est importante car être connecté au broker ne garantit pas que l'usine est prête à produire.

15. Gestion des statuts et numéros de série

La gestion des statuts est importante car elle permet de rendre visible un traitement qui se déroule en arrière-plan. Lorsqu'une commande est envoyée, l'utilisateur ne peut pas voir directement ce que fait le backend. Les statuts servent donc d'indicateurs intermédiaires. Ils transforment des événements techniques en informations compréhensibles

A. États de traitement

Le sujet propose en bonus l'envoi d'événements de progression. Cette fonctionnalité est présente. Une commande correcte passe par une validation, puis un statut processing pendant l'appel à l'usine, puis un statut processed lorsque les lunettes ont été produites. Le message delivery termine la transaction réussie.

Étape	Topic	Charge utile
Validation	orders/{id}/validated	Vide
Début de production	orders/{id}/status	processing
Fin de production	orders/{id}/status	processed
Livraison	orders/{id}/delivery	Types et numéros de série

Dans une production mutualisée, chaque commande conserve son propre cycle visible. Même si l'usine fabrique les produits dans un lot commun, le serveur publie les statuts et la livraison sur le topic portant l'identifiant de la commande concernée.

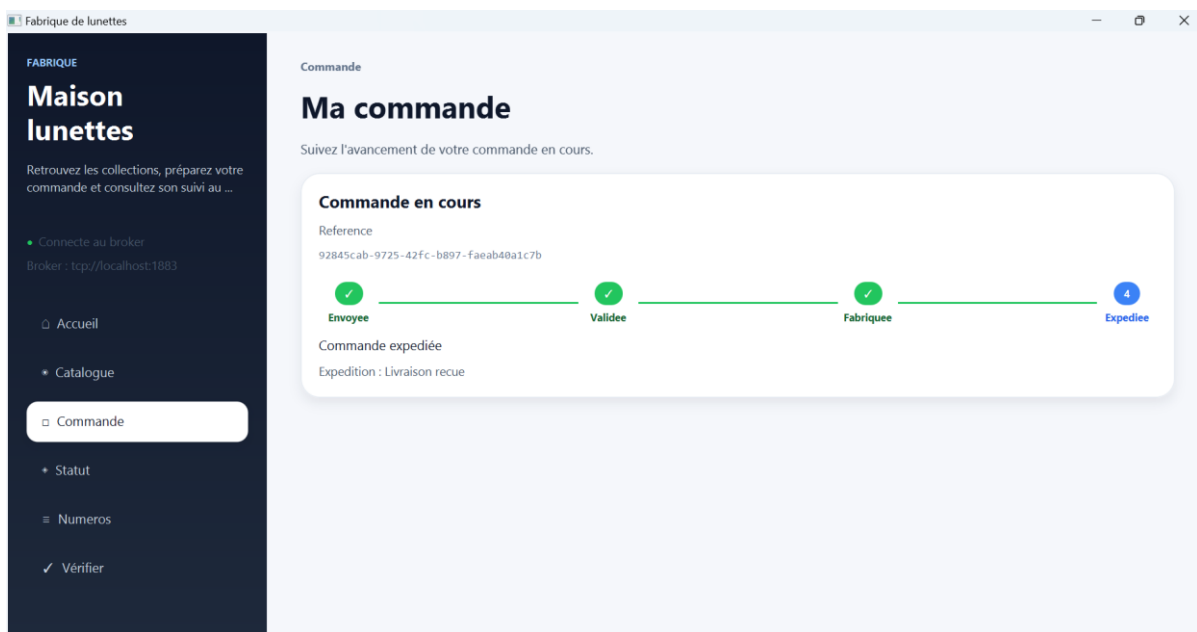


Figure 6 - Timeline de suivi de commande après fabrication et livraison.

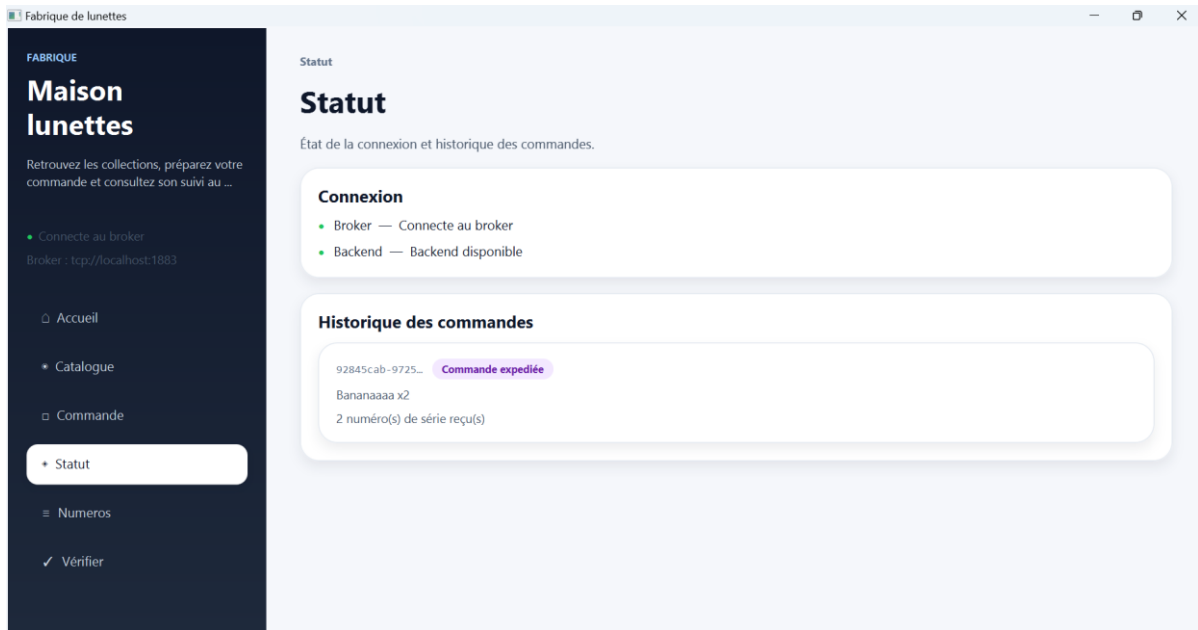


Figure 7 - Écran de statut avec connexion au broker et disponibilité backend.

B. Annulation et erreur de fabrication

Deux situations mettent fin à une transaction sans livraison. Une commande invalide est refusée avant l'appel à l'usine. Une commande valide peut également échouer pendant la production si fabricant déclenche une exception. Dans ce second cas, le serveur publie un événement error afin d'informer le frontend.

Situation	Message publié	Suite du traitement
Commande incorrecte	orders/{id}/cancelled	Aucune fabrication
Erreur pendant produire	orders/{id}/error	Aucune livraison réussie
Production terminée	orders/{id}/delivery	Transaction clôturée avec succès

C. Numéros de série

Après la fabrication, chaque objet `Fabricateur.Lunette` expose son type et son numéro de série. `MqttServer` ajoute ces informations dans un registre en mémoire, représenté par une `ConcurrentHashMap`. Le choix d'une structure concurrente est cohérent avec le traitement de plusieurs commandes, car des productions peuvent se terminer tandis qu'une vérification est demandée.

Une vérification de numéro de série suit un échange simple. Le frontend publie sur `serials/{numero}/check`. `MqttServer` consulte le registre et publie le type de lunette si le numéro existe ou la valeur `invalid` dans le cas contraire.

Événement client	Événement backend	Exemple de réponse
<code>serials/SN-001/check</code>	<code>serials/SN-001</code>	BANANA
<code>serials/NUMERO-ABSENT/check</code>	<code>serials/NUMERO-ABSENT</code>	invalid

Les captures suivantes montrent l'état de la commande, l'historique, les numéros de série et l'écran de vérification. Elles confirment que l'interface reçoit bien les informations produites par le backend après la fabrication.

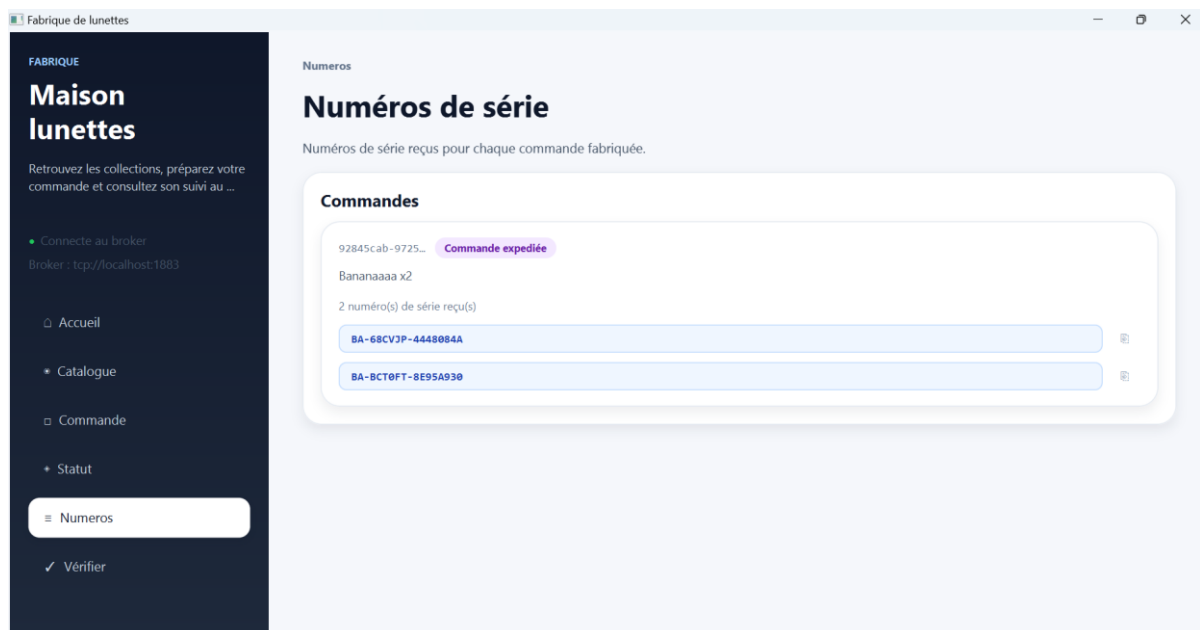


Figure 8 - Affichage des numéros de série reçus après fabrication.

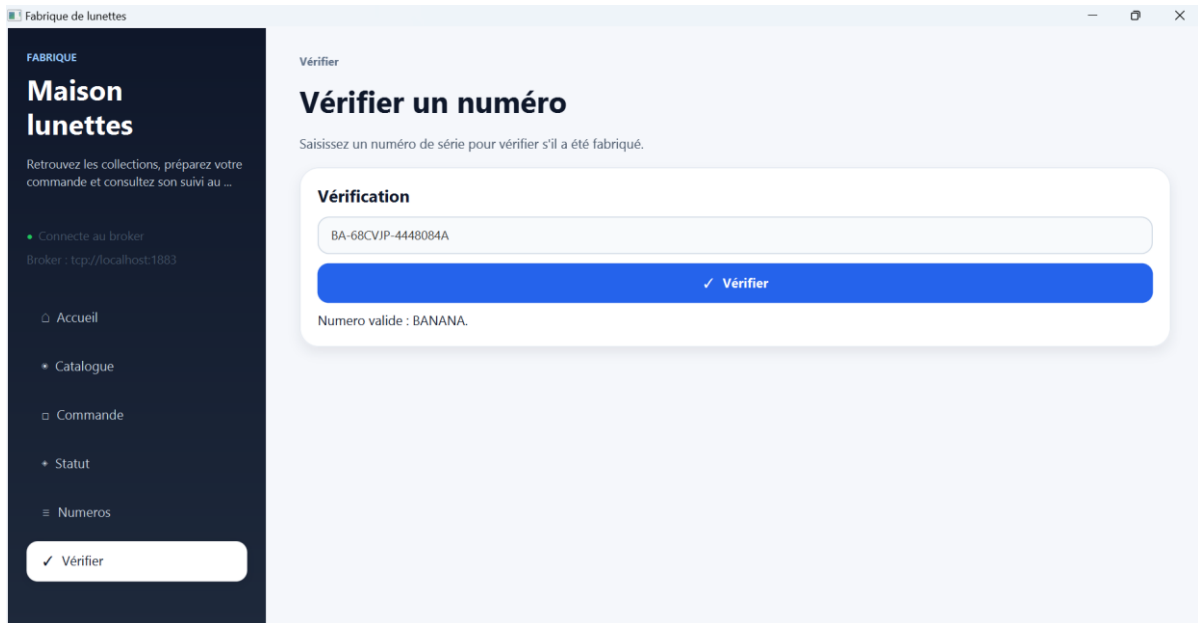


Figure 9 - Vérification d'un numéro de série valide.

D. Cohérence entre interface et backend

La cohérence entre les statuts backend et l'affichage frontend est essentielle. Si le backend publie `processing`, l'interface doit l'interpréter comme une fabrication en cours. Si le backend publie `processed`, l'interface doit montrer que la fabrication est terminée. Le vocabulaire utilisé dans les payloads doit donc être compris des deux côtés. Cette cohérence fait partie du contrat de communication.

Les numéros de série sont également liés à cette cohérence. Ils ne doivent être affichés qu'après la livraison. Cela évite de montrer des informations incomplètes ou anticipées. L'écran des numéros devient donc la dernière étape visible du processus de fabrication. Il donne une preuve concrète que la commande a abouti.

16. Tests et validation

Les tests ont pour objectif de vérifier que certaines parties du projet fonctionnent de manière fiable. Dans notre projet, les tests unitaires couvrent principalement le module partagé et certaines règles de validation backend. Ils permettent de s'assurer que les commandes sont correctement interprétées, que les quantités sont cohérentes et que le format des messages est traité correctement.

Les tests du module partagé vérifient notamment le calcul des quantités et le comportement du formateur de messages. Cette partie est importante car une erreur d'encodage ou de décodage pourrait empêcher le backend de comprendre les commandes envoyées par le frontend. Tester ces classes permet de sécuriser le contrat entre les deux applications.

Les tests backend portent sur la validation des commandes. Ils vérifient par exemple les commandes vides, les quantités nulles, les quantités négatives ou les quantités trop élevées. Ces tests sont nécessaires car le backend doit rester responsable de la cohérence métier. Même si le frontend limite les entrées utilisateur, un message MQTT pourrait être envoyé directement par un autre client ou par un outil de test.

La validation fonctionnelle a aussi été réalisée manuellement à travers l'interface. Le parcours testé est le suivant : lancement du broker, lancement du backend, lancement du frontend, connexion au broker, sélection d'un produit dans le catalogue, envoi de la commande, suivi de la timeline, réception des numéros de série et vérification d'un numéro. Les captures insérées dans ce rapport montrent ce parcours côté interface.

Cependant, la couverture de tests reste perfectible. Il manque de vrais tests d'intégration MQTT, des tests JavaFX automatisés et un test de bout en bout allant de la commande jusqu'à la livraison. Cette limite ne signifie pas que le projet ne fonctionne pas, mais elle montre que la validation pourrait être renforcée. Dans une version plus professionnelle, ces tests seraient nécessaires pour sécuriser les évolutions futures.

A. Commandes de test

Les commandes de test prévues sont `mvn test` pour l'ensemble du projet, `mvn -pl lunettes-shared test` pour le module partagé et `mvn -pl lunettes-backend test` pour le backend. Le résultat attendu est le passage des tests JUnit lorsque les accès Maven nécessaires sont disponibles. Les captures des résultats Maven peuvent être ajoutées en annexe afin de renforcer la preuve de validation.

B. Différence entre tests unitaires et tests d'intégration

Les tests unitaires vérifient une classe ou une méthode isolée. Ils sont rapides et utiles pour sécuriser des règles simples. Par exemple, tester le formateur de messages permet de vérifier que l'encodage et le décodage fonctionnent dans plusieurs cas. Ces tests sont importants car ils détectent rapidement des erreurs de logique.

Les tests d'intégration vérifient plusieurs composants ensemble. Dans notre projet, un test d'intégration complet devrait lancer ou simuler un broker MQTT, envoyer une commande, vérifier que le backend la reçoit, déclenche la fabrication et publie une livraison. Ce type de test est plus complexe, mais il donnerait une preuve plus forte du fonctionnement global.

Les tests manuels restent utiles pour vérifier l'ergonomie et le parcours utilisateur. Ils permettent de constater que les écrans sont lisibles, que les boutons fonctionnent et que la

timeline évolue correctement. En revanche, ils ne remplacent pas les tests automatisés, car ils dépendent de l'attention de la personne qui teste.

17. Packaging Maven

Le packaging Maven est une partie importante du projet, car il permet de transformer le code en livrables exécutables. Un projet qui fonctionne uniquement depuis l'IDE reste difficile à partager. Avec Maven, on peut compiler, tester et packager les modules avec des commandes standardisées. Cela facilite le lancement par d'autres personnes et répond à la demande du cahier des charges.

Le backend est packagé avec ses dépendances afin de produire un JAR exécutable. Cette approche est pratique car elle évite à l'utilisateur de devoir gérer manuellement toutes les bibliothèques nécessaires. Le JAR backend contient la classe principale permettant de démarrer le serveur. Il peut ainsi être lancé après le démarrage du broker MQTT.

Le frontend est également packagé en JAR. Il contient l'application JavaFX et les ressources nécessaires à l'interface. Le lancement du frontend doit se faire après la mise en route du broker et, idéalement, du backend. L'interface peut indiquer l'état de connexion et la disponibilité du serveur, ce qui aide l'utilisateur à comprendre si le système est prêt.

Le module partagé n'est pas une application autonome. Il est utilisé comme dépendance par le frontend et le backend. Son packaging permet de le rendre disponible aux autres modules Maven. Cette séparation facilite la compilation et évite la duplication des classes communes.

Les commandes Maven principales sont `mvn clean compile` pour compiler, `mvn test` pour exécuter les tests et `mvn clean package` pour produire les artefacts. Dans le cadre d'une release, le workflow peut utiliser `mvn clean deploy` afin de publier les artefacts dans un dépôt Maven. Cette chaîne montre que le projet est pensé pour être livré et pas seulement exécuté localement.

A. Intérêt du packaging pour le projet

Le packaging est une preuve de maturité du projet. Il permet de rendre le backend et le frontend indépendants de l'environnement de développement. Un utilisateur peut récupérer les JARs, démarrer Mosquitto, lancer le backend puis lancer l'interface. Cette logique correspond mieux à une livraison logicielle réelle.

B. Reproductibilité du build

La reproductibilité du build signifie que le projet peut être reconstruit de la même façon sur plusieurs environnements. Maven aide à atteindre cet objectif car les versions des dépendances et les plugins sont déclarés dans les fichiers `pom.xml`. Un nouvel utilisateur n'a pas besoin de chercher manuellement les bibliothèques à ajouter au classpath.

Cette reproductibilité est particulièrement utile dans un projet multi-modules. Le module partagé doit être compilé avant les modules qui l'utilisent. Maven sait gérer cet ordre. Il permet donc de construire l'ensemble du projet avec une commande globale, ce qui réduit les manipulations manuelles et les erreurs.

Le packaging en JAR rend aussi le projet plus simple à présenter. Au lieu d'ouvrir l'IDE et de lancer plusieurs classes manuellement, on peut fournir des artefacts exécutables. Cela correspond mieux à la logique d'un livrable attendu.

18. CI/CD avec GitHub Actions

La CI/CD a été mise en place avec GitHub Actions. L'objectif est d'automatiser la génération et la publication des livrables au moment d'une release. Cette automatisation évite de produire les JARs manuellement sur un poste de développement. Elle rend la livraison plus reproductible, car le même workflow est exécuté à chaque publication.

Le workflow configure un environnement Java 21 avec Temurin. Il crée également un fichier `settings.xml` contenant les accès nécessaires à GitHub Packages et à la dépendance du fabricant. Cette étape est importante car le backend dépend du fabricant fourni, qui n'est pas une dépendance Maven publique classique. Sans configuration correcte des tokens, la compilation pourrait échouer.

Le workflow lance ensuite une commande Maven de déploiement et ajoute les JARs générés aux assets de la release. Cela permet de retrouver directement les livrables dans GitHub, sans devoir cloner le projet et compiler localement. Pour un utilisateur final ou pour l'évaluateur, cette organisation est pratique car elle centralise le code source et les exécutables.

Cette chaîne CI/CD répond à une partie de l'exigence production ready. Elle montre que le projet prend en compte la livraison, la publication et la reproductibilité. Cependant, elle peut être améliorée. D'après l'analyse du dépôt, le workflow utilise l'option `-DskipTests`. Cela signifie que les tests ne bloquent pas la publication. Dans un projet professionnel, il serait préférable d'exécuter les tests automatiquement avant de publier une release.

Une amélioration possible serait de créer deux workflows. Le premier s'exécuterait à chaque push ou pull request et lancerait les tests. Le second serait déclenché lors d'une release et publierait les JARs uniquement si les tests sont passés. On pourrait aussi ajouter une matrice d'environnements, des logs plus détaillés et une vérification du lancement des JARs. Ces améliorations renforceraient la fiabilité de la livraison.

A. Relation entre CI/CD et qualité

La CI/CD n'est pas seulement un outil de publication. Elle fait partie de la qualité logicielle. Si elle exécute les tests, vérifie le packaging et publie les artefacts, elle réduit le risque de livrer une version cassée. Dans notre projet, la base existe déjà. Le principal axe d'amélioration serait d'intégrer davantage les tests dans cette chaîne.

B. Publication et secrets

La publication Maven via GitHub Actions nécessite des accès. Le workflow doit pouvoir récupérer les dépendances privées et publier les artefacts. Les tokens et secrets GitHub permettent de ne pas écrire ces informations directement dans le code. C'est une pratique importante, car les identifiants ne doivent pas être visibles dans un dépôt public.

Cette dépendance aux secrets peut aussi être une fragilité. Si un secret expire, est mal nommé ou n'est pas disponible dans un fork, le workflow peut échouer. Il faut donc documenter clairement les secrets nécessaires et prévoir une solution de secours pour compiler localement. Cette documentation est une partie souvent oubliée mais importante du DevOps.

La release GitHub a l'avantage de regrouper les livrables. L'évaluateur peut retrouver les JARs associés à une version précise du code. Cela évite de mélanger une version du code avec des exécutables produits à un autre moment. La cohérence entre code source et artefacts est un point positif pour la traçabilité du projet.

19. Difficultés rencontrées

Le projet combine des contraintes de communication, de concurrence et de production. La difficulté ne réside pas uniquement dans l'envoi d'un message MQTT, mais dans la conservation d'une transaction cohérente du début de la commande jusqu'à la réception des numéros de série.

La première difficulté a été de bien comprendre la logique asynchrone. Dans un développement plus classique, on pense souvent en termes d'appel direct : l'utilisateur clique, une méthode est appelée, puis une réponse est retournée. Avec MQTT, le raisonnement est différent. L'utilisateur déclenche un événement, puis les réponses arrivent plus tard sur d'autres topics. Cela demande de gérer l'état de l'interface et de ne pas supposer que tout arrive immédiatement.

La deuxième difficulté a été la définition du protocole de communication. Il fallait choisir les topics, le format des payloads et les règles de réponse. Une mauvaise convention aurait rendu le système confus. Par exemple, si tous les statuts étaient publiés sur un topic unique, il aurait été plus difficile de suivre plusieurs commandes. L'utilisation de l'identifiant de commande dans les topics a permis de clarifier les échanges.

La troisième difficulté a concerné l'intégration du fabricant fourni. Comme il s'agit d'une dépendance externe, il faut respecter ses types, sa capacité et ses méthodes. L'usine a été conçue pour servir d'intermédiaire entre notre modèle de commande et le modèle attendu par le fabricant. Cette couche d'adaptation a demandé une attention particulière afin que la fabrication corresponde bien aux quantités demandées.

La quatrième difficulté a été la gestion de l'interface JavaFX. L'application devait rester compréhensible tout en affichant plusieurs informations état du broker, disponibilité du backend, catalogue, panier, statut, historique et numéros de série. Il fallait donc organiser les écrans de manière logique. Le menu latéral a permis de rendre la navigation plus claire.

La cinquième difficulté a été la livraison du projet. Créer un projet Maven multi-modules, produire des JARs et configurer GitHub Actions demande plus de rigueur qu'un projet simple. Les dépendances, les classes principales, les plugins Maven et les secrets GitHub doivent être correctement configurés. Cette partie est moins visible que l'interface, mais elle est essentielle pour produire un livrable propre.

Enfin, une difficulté transversale a été la coordination entre les membres. Le frontend dépend du protocole défini avec le backend. Le backend dépend de l'usine. L'usine dépend du fabricant. Le DevOps dépend de la structure Maven. Chaque choix technique avait donc un impact sur les autres parties. Les échanges réguliers ont permis de limiter les incohérences et de faire évoluer le projet de manière progressive.

A. Apports de ces difficultés

Ces difficultés ont eu un intérêt pédagogique. Elles ont montré qu'un projet logiciel ne se limite pas à écrire du code qui compile. Il faut aussi penser l'architecture, les échanges, les erreurs, les tests, la livraison et la maintenance. Cette prise de recul est l'un des principaux apprentissages du projet.

B. Gestion du temps et intégration finale

Comme souvent dans un projet de groupe, l'intégration finale est une étape sensible. Des parties développées séparément doivent fonctionner ensemble. Un écran peut être prêt, mais attendre un topic différent de celui publié par le backend. Une classe backend peut fonctionner seule, mais ne pas recevoir le format exact envoyé par le frontend. L'intégration oblige donc à vérifier les détails.

La gestion du temps a aussi demandé de prioriser. Les fonctionnalités obligatoires ont été traitées en priorité commande, fabrication, livraison, numéros de série et vérification. Les améliorations plus avancées, comme la persistance ou la sécurité MQTT, ont été identifiées mais laissées comme pistes futures. Cette priorisation est cohérente avec un projet universitaire limité dans le temps.

20. Limites techniques et améliorations possibles

Même si les fonctionnalités principales sont présentes, le projet possède plusieurs limites techniques. Il est important de les présenter clairement, car un bon rapport ne doit pas seulement valoriser ce qui fonctionne. Il doit aussi montrer les points qui pourraient être améliorés. Ces limites ne remettent pas en cause le fonctionnement global de la démonstration, mais elles indiquent ce qu'il faudrait renforcer pour une version plus robuste.

La première limite concerne la sécurité MQTT. Le broker est utilisé localement sur localhost:1883, sans authentification forte, sans TLS et sans règles d'accès avancées. Pour un projet pédagogique local, cela reste acceptable. En revanche, dans un contexte réel, il faudrait protéger les échanges. Une commande ou un numéro de série peut être une donnée importante. Il faudrait donc ajouter une authentification, chiffrer les communications et limiter les topics accessibles à chaque client.

La deuxième limite concerne la persistance. Les commandes et les numéros de série sont conservés en mémoire. Cela permet de faire fonctionner le projet pendant une session, mais les données disparaissent au redémarrage du backend. Dans une entreprise, ce comportement serait problématique. Il faudrait enregistrer les commandes, les statuts et les numéros de série dans une base de données. Cela permettrait de retrouver l'historique, de vérifier un numéro ancien et de reprendre le traitement après incident.

La troisième limite concerne la gestion des erreurs réseau. Le frontend gère mieux la reconnexion que le backend, mais l'ensemble pourrait être renforcé. Si le broker tombe, si le backend redémarre ou si un message est perdu, l'application devrait avoir une stratégie claire. On pourrait utiliser des sessions MQTT persistantes, des niveaux de qualité de service plus adaptés et des messages de reprise. Cela rendrait le système plus fiable.

La quatrième limite concerne le format des messages. Le format textuel maison est simple et lisible, mais il reste fragile. Une virgule ou deux-points mal placés peuvent rendre le message invalide. Un format JSON ou Protobuf permettrait de structurer les données, d'ajouter des champs sans casser le protocole et de mieux gérer les erreurs de validation. Pour une première version, le format actuel est suffisant, mais il serait limité dans une application plus large.

La cinquième limite concerne les tests. Les tests unitaires existants sont utiles, mais ils ne couvrent pas tout le système. Il manque des tests d'intégration MQTT, des tests de bout en bout et des tests automatisés de l'interface JavaFX. Il faudrait aussi intégrer les tests dans la CI/CD. Cela éviterait de publier une release sans vérifier automatiquement le fonctionnement des éléments principaux.

Enfin, la structure du frontend pourrait être améliorée. La classe ClientShell concentre beaucoup de responsabilités : navigation, création des écrans, interactions et mise à jour de l'état. Cette approche fonctionne pour un projet de taille limitée, mais elle peut devenir difficile à maintenir. Une future version pourrait séparer les contrôleurs, les vues et les services afin de mieux organiser le code.

A. Améliorations prioritaires

Si nous devons poursuivre le projet, les priorités seraient la persistance des données, l'amélioration des tests, l'exécution des tests en CI/CD et la sécurisation MQTT. Ces quatre améliorations apporteraient un gain direct en robustesse. Ensuite, on pourrait travailler l'ergonomie de l'interface, la supervision du backend et l'extension de l'usine à plusieurs fabricateurs.

21. Conclusion

Le projet La Fabrique de Lunettes nous a permis de concevoir un système distribué complet autour d'un cas métier concret. L'application ne se limite pas à une interface graphique : elle combine un frontend JavaFX, un backend Java, une usine de fabrication, un module partagé, une communication MQTT, une structure Maven et une chaîne GitHub Actions. Cette combinaison donne un projet cohérent, proche des problématiques rencontrées dans des architectures logicielles réelles.

Les fonctionnalités principales du cahier des charges ont été implémentées. L'utilisateur peut accéder à un écran d'accueil, consulter un catalogue, choisir des quantités, passer une commande, suivre son état, recevoir des numéros de série et vérifier un numéro. Le backend reçoit les commandes, les valide, déclenche l'usine, récupère les numéros de série et publie les messages nécessaires au frontend. Le système fonctionne donc de bout en bout.

Le choix de MQTT et du publish/subscribe a été pertinent pour ce projet. Il a permis de découpler le frontend du backend et de représenter correctement le caractère asynchrone de la fabrication. Le client ne bloque pas en attendant une réponse immédiate. Il reçoit des événements au fur et à mesure de l'avancement. Cette logique demande plus de conception qu'un appel direct, mais elle donne une architecture plus souple.

Le projet a aussi montré l'importance de la qualité et de la livraison. Maven a permis de structurer les modules et de gérer les dépendances. Les tests unitaires ont sécurisé une partie des règles métier et du format de messages. GitHub Actions a apporté une première automatisation de la publication. Ces éléments ne sont pas seulement techniques : ils montrent une manière plus professionnelle de construire et de livrer une application.

Enfin, le projet possède des limites qui ouvrent des pistes d'amélioration. La persistance des commandes, la sécurité MQTT, les tests d'intégration, la CI/CD avec tests obligatoires et une meilleure séparation du frontend seraient des évolutions importantes. Ces limites ne doivent pas être vues comme un échec, mais comme la suite logique d'un projet qui fonctionne déjà dans un cadre pédagogique. Elles montrent que nous sommes capables d'analyser notre propre solution avec recul.

En conclusion, ce projet a été formateur car il a réuni plusieurs dimensions : programmation Java, interface graphique, communication réseau, architecture, concurrence, packaging, tests et DevOps. Il nous a obligés à penser l'application comme un système complet, où chaque composant doit collaborer avec les autres. C'est cette vision globale qui constitue l'apport principal du travail réalisé.

A. Bilan personnel et collectif

Sur le plan collectif, le projet a demandé une collaboration entre des profils et des tâches différentes. Chacun a travaillé sur un domaine, mais le résultat dépendait de l'intégration de tous les domaines. Cette expérience montre que le développement logiciel est aussi une activité de communication. Les choix de topics, de formats et de responsabilités doivent être compris par toute l'équipe.

Sur le plan technique, le projet nous a permis de mieux comprendre la différence entre une application locale et un système distribué. Même avec un broker local, les problèmes changent : il faut gérer les messages, les statuts, les erreurs, la disponibilité des services et la livraison

des artefacts. Cette complexité supplémentaire rend le projet plus proche d'un cas professionnel.

Le résultat final est donc satisfaisant pour un projet pédagogique. Il présente une chaîne complète, des fonctionnalités visibles, une architecture identifiable et une réflexion sur les limites. Le rapport permet de documenter cette démarche et de montrer que les choix techniques ne sont pas seulement des lignes de code, mais des réponses à des contraintes précises.

Annexes

Les annexes regroupent les éléments pratiques liés à l'exécution et à la validation du projet. Elles peuvent être complétées avec des captures supplémentaires, notamment les logs du backend, les résultats Maven et la page GitHub Actions réussie. Les captures de l'interface ont déjà été intégrées dans le corps du rapport afin d'illustrer le parcours utilisateur.

Annexe

Topics MQTT principaux

Les topics ci-dessous représentent le protocole de communication principal. Ils peuvent être utilisés pour comprendre le fonctionnement du système ou pour réaliser des tests manuels avec un client MQTT.

<code>orders/{orderId}</code>	-> commande envoyée par le frontend
<code>orders/{orderId}/validated</code>	-> validation de la commande par le backend
<code>orders/{orderId}/cancelled</code>	-> annulation de la commande
<code>orders/{orderId}/status</code>	-> statut de fabrication
<code>orders/{orderId}/delivery</code>	-> livraison des numéros de série
<code>orders/{orderId}/error</code>	-> erreur de traitement
<code>serials/{serial}/check</code>	-> demande de vérification d'un numéro
<code>serials/{serial}</code>	-> réponse à la vérification
<code>backend/ping</code>	-> test de disponibilité backend
<code>backend/pong</code>	-> réponse du backend